

OctoNav

OctoNav (*student proposal*)

Assessment: ✓ Fit · **Category:** Immersive Experience · **Assessed:** 2026-09-21 · **Status:** awaiting instructor approval · **Runner-up category:** Visualization & Analytics

Proposal (as submitted): An Android app (Android Studio) providing **indoor navigation for SIT Block E2** with a **real-time 3D / isometric campus view**. Users pick a destination and a mobility preference; the system computes an **optimal multi-floor route** weighted by distance, turn count and accessibility. Campus geometry is captured via **dedicated LiDAR walk-throughs**; users can report temporary obstructions in real time, feeding **live rerouting**. The app has login (Google / school email), bottom-tab navigation, a user profile and a local database for search history and saved data. Named technical features: custom **2.5D and 3D visualization** of Block E2; a **cloud database** to share data between devices in real time; a **nodes-to-model** step that turns LiDAR readings into working models; and a **pathfinding engine** searching across three axes (X, Y, Z). Core-requirement notes: a **frame-bounded C++ loop** that each tick pulls in new/buffered LiDAR data, updates the internal structure, and renders the current state to a 2.5D and a 3D viewport. Tools: a **raw-sensor** → **point-cloud conversion tool** and an **engine inspector** (query point attributes; visual debug overlays for octree bounds). Design features: real-time crowdsourced updates on closures/obstacles; dynamic routing on live crowd densities; in-depth 1:1 LiDAR mapping; and accessibility (wheelchair / physically-handicapped) **conditional routing**.

Note on category: filed under **Immersive Experience** — the deliverable is an explorable, real-time 3D/2.5D reconstruction of a real building navigated with the team's own engine, matching the category's "*virtual campus tour*" exemplar. Runner-up **Visualization & Analytics** (the live crowd-density / route data as moving data) — but OctoNav is a navigable spatial world, not a dashboard, so Immersive Experience is the better home. (Not GovTech: there is no staff/operations side or roles/audit trail.)

Scope & risk note (advisory, not a blocking condition): LiDAR capture → point-cloud → nodes-to-model → octree → 3-axis pathfinding → real-time 3D render → live crowdsourced rerouting is a **large, ambitious surface**, and this is a genuine **3D** project (the "strong students only" row) — the LiDAR/point-cloud → model pipeline is the key risk. Scope the floor to **one or two floors of Block E2** from a curated capture, with the C++ loop, the octree + 3-axis pathfinding and the 2.5D/3D view proven; treat the full-building capture, live crowd-density rerouting and cloud real-time sync as the stretch.

References:

- [Google Maps Indoor Maps](#) — Google (official indoor-mapping + multi-floor "Live View" indoor navigation that understands altitude/floors — the multi-floor indoor-wayfinding precedent)
- [Matterport](#) — "A Tactical Guide to LiDAR Room Mapping" (LiDAR walk-throughs turned into an accurate, **navigable 3D digital twin** of a building — the LiDAR-capture → navigable-3D-model precedent)
- [Mappedin](#) — "Accessible indoor navigation software" (indoor wayfinding with **barrier-free / wheelchair routing** that prioritizes elevators and avoids stairs — the accessibility conditional-routing precedent)

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

Core Tech Requirement	OctoNav
External Database	✓
Authentication »	✓
Analytics »	●
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	✓
2D Graphical Output	✓
3D (strong students only)	✓
Networking System	●

How each requirement applies:

Platform fit: **Main:** Native mobile (Native Android — the stated build; device: Pixel 9a) — a touch app with a real-time 2.5D/3D campus view and a bottom-tab shell; the Native PC build is the development/tools vehicle (the conversion tool + engine inspector). **Extended:** Firebase (cloud DB for real-time cross-device sharing of obstruction reports + crowd density; Google/school-email auth) + a Linux server the team sets up by M2 (full use T2) for the LiDAR point-cloud processing pipeline.

Front end: A real-time renderer of Block E2 in both a ****2.5D**** and a ****3D**** viewport (multi-floor geometry, the computed route highlighted, obstruction

markers, the mobility-preference and destination UI, bottom-tab nav, profile).

Languages: Native — Kotlin Native Android UI + the C++ engine via JNI (Android Studio); OpenGL ES for the realtime 2.5D/3D drawn in a GLSurfaceView; Web/desktop — SDL2/3 + OpenGL ES for the PC dev/tools build (conversion tool + inspector).

Back end: The frame-bounded C++ loop (ingest LiDAR/point-cloud data → update the octree spatial structure → render both viewports) + the **3-axis (X,Y,Z) multi-floor pathfinding** weighted by distance/turns/accessibility + live rerouting from crowd/obstruction data; plus the cloud DB for real-time sharing and the LiDAR point-cloud processing on the server. **Languages:** C++ (loop + pathfinding); Firebase-backed services + the server pipeline on the Extended tier.

Backend scope: Server-indicated (T2) — the cloud real-time-sync + LiDAR-processing seam set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* the C++ loop is load-bearing — each tick it pulls in new/buffered LiDAR/point-cloud data, updates the octree spatial structure, runs any pending 3-axis pathfinding/rerouting, and renders the current state to the 2.5D and 3D viewports.
- *Data-driven:* the campus geometry (from the nodes-to-model step), node/edge graph with accessibility attributes, room/destination metadata and routing weights are all data, authored/converted via the tools — a new floor or obstruction is a data change.
- *2D:* the 2.5D viewport, the route/turn overlays, the obstruction markers and the app UI (bottom-tabs, profile, search) are custom 2D rendering — satisfying the 2D gate. *3D:* ✓ **central** — the real-time 3D campus view + the octree/3-axis spatial work (strong-students; the key risk). *Networking:* the real-time cross-device sharing of reports/crowd density is via the cloud DB (Application Communication ✓), so a bespoke real-time Networking System is optional (●).
- *Custom tool:* the **conversion tool** (raw LiDAR/sensor input → the engine's internal point-cloud format) **and** the **engine inspector** (query point attributes, octree-bounds debug overlays).
- *External DB:* the cloud DB (real-time reports/crowd density, cross-device) + the local DB (search history, saved data). *Auth* »: Google / school-email login. *Analytics* »: optional — common routes / bottleneck reporting.

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Immersive Experience pool in [milestone-goals-pool-per-category.pdf](#) (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic**

Tech & dependencies

- **Tech: Web build:** SDL2/3 window + the frame-bounded C++ loop (ingest LiDAR/point-cloud → update the octree → render both viewports), plus the bottom-tab app shell and destination/mobility input; C++17/20, CMake, Git. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2.5D/3D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; the octree (M1G8) and pathfinding (M1G12) run on its tick.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**

Tech & dependencies

- **Tech:** Data model in C++ for the campus geometry, the node/edge navigation graph with accessibility attributes, room/destination metadata and routing weights, in JSON/serialized form, hot-reloadable — a new floor or obstruction is a data change.
- **Prerequisites:** G1 (loop & update hooks).
- **Feeds into:** M2G2 (the conversion/editor tools), M1G12 (pathfinding graph), M2G12 (cloud sync of reports).

- **G3 ★ Debugging, Profiling, Stress Test & CI**

Tech & dependencies

- **Tech:** ImGui debug overlays (octree bounds, point attributes — the basis of the inspector tool), Tracy profiling of the ingest→update→render→pathfind budget on a full-floor point cloud, Catch2 tests for the pathfinding and data loading, on a GitHub Actions Windows+Android matrix.

- **Prerequisites:** G1 (loop to instrument).
- **Feeds into:** M2G3 (CI hardening); the on-device budget M2G1 validates; the engine inspector.
- **G4 2D Scene Rendering & Animation System** (extended to **2.5D + 3D**) — the dual-viewport campus renderer is the whole read of the app.
Tech & dependencies
 - **Tech: Web build:** A renderer with a 2.5D (isometric) and a 3D viewport of the multi-floor campus (camera, lit geometry, the highlighted route, obstruction markers), batching, depth; OpenGL ES, GLSL, FreeType HUD. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2.5D/3D is needed — no SDL on native, some screens are pure Native UI.
 - **Prerequisites:** G1 (loop & window), G8 (geometry to draw).
 - **Feeds into:** M2G4 (Rendering II — 3D shading/lighting per floor); every visible frame.
- **G8 Scene Graph & Content System** — the **octree spatial structure** built from the LiDAR nodes-to-model step that holds the campus geometry.
Tech & dependencies
 - **Tech:** An octree/scene structure in C++ holding the point-cloud-derived geometry and the walkable node graph, with the nodes-to-model conversion that turns LiDAR readings into the working model; spatial queries for rendering + pathfinding.
 - **Prerequisites:** G1 (loop), G2 (data model).
 - **Feeds into:** M1G4 (what it renders), M1G12 (what it paths over), the inspector/conversion tools.
- **G12 Map / Navigation System** — the **3-axis (X,Y,Z) multi-floor pathfinding** weighted by distance, turns and accessibility, with rerouting.
Tech & dependencies
 - **Tech:** A* / weighted graph search across X/Y/Z (multi-floor, via stairs/lifts), cost weights for distance, turn count and accessibility (conditional routing for wheelchair users), and a rerouting path that responds to obstruction/crowd updates.
 - **Prerequisites:** G1 (tick), G8 (the node graph).
 - **Feeds into:** The route the renderer draws; M2G12 (live crowd/obstruction data drives rerouting).

M2:

- **G1 ★ Architecture — Optimization & Platform Validation**

Tech & dependencies

- **Tech: Web build:** Tracy profiling of the ingest→update→render→pathfind loop on a full floor, ASan on MSVC, memory pools for the point cloud/octree, and on-device validation (Pixel 9a); the Firebase/Linux-server seam (cloud sync + LiDAR processing) set up (account + software + basic connectivity test) by M2. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2.5D/3D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** All M1 goals (hardens the M1 codebase).
- **Feeds into:** The Native-Android deliverable + a stable budget on a real Block E2 capture.

- **G2 ★ Content Editor — Core (Conversion Tool + Scene/Node Editor + Inspector)** — the raw-sensor → point-cloud conversion tool, a scene/node editor, and the engine inspector (point attributes + octree-bounds overlays).

Tech & dependencies

- **Tech:** Dear ImGui + the shared C++ renderer: convert raw LiDAR/sensor input into the internal point-cloud format, place/adjust nav nodes and accessibility attributes, and inspect point attributes / octree bounds, with round-trip serialization and live preview.
- **Libraries/Software:** Dear ImGui, nlohmann/json, the project's OpenGL ES renderer (shared with the app).
- **Prerequisites:** G2 (data model), M1G8 (octree), M1G4 (renderer).
- **Feeds into:** All campus content; the custom-tools showcase.

- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, reproducible Windows + Android Debug builds, tests (pathfinding, conversion, data loading) and packaging via GitHub Actions. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2.5D/3D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G3 (CI & test foundation).

- **Feeds into:** Release/showcase builds of every goal.
 - **G4 Rendering II** — 3D shading, lighting and post-FX for the multi-floor campus view.
- Tech & dependencies
- **Tech:** OpenGL ES / GLSL lighting and materials for the 3D floors, framebuffers/post-FX, and clear floor-transition visuals so multi-floor routes read.
 - **Prerequisites:** M1G4 (base renderer), M2G1 (optimization).
 - **Feeds into:** A legible, polished 2.5D/3D campus at showcase.
 - **G12 Firebase — Analytics & Content Delivery** — the cloud DB for real-time cross-device sharing of obstruction reports + crowd density (feeding live rerouting), accounts and analytics.

Tech & dependencies

- **Tech:** Firebase Auth (Google/school email), Firestore/Realtime DB for crowdsourced obstruction reports + crowd density synced across devices in real time, and optional Analytics, via the REST API from C++.
- **Prerequisites:** G2 (report/route data model), M1G12 (rerouting consumes it).
- **Feeds into:** Live rerouting; the crowdsourced updates feature.
- **G16 Accessibility & Comfort Options** — the wheelchair / physically-handicapped **conditional routing** + accessibility UI/comfort options.

Tech & dependencies

- **Tech:** Accessibility routing profiles (avoid stairs, prefer lifts/ramps) driving the M1G12 weights, plus UI accessibility (contrast, type scale, reduced motion) and clear mobility-preference selection.
- **Prerequisites:** M1G12 (weighted pathfinding), M2U1/U2 (UI).
- **Feeds into:** The accessibility design feature; inclusive wayfinding at showcase.

Track Goals & Team Composition

Recommended composition: 6 CS + 2 Design + 1 Art (9) — because the campus **geometry is LiDAR-captured rather than hand-modelled**, the art surface is smaller than a typical Immersive Experience (visual style, materials/lighting, wayfinding iconography and UI), so one artist can carry it; the **wayfinding UX is the larger surface** (destination/mobility selection, multi-floor route legibility, the crowdsourced-report flow, accessibility conditional routing, onboarding), so two designers own the design + UI/UX (rules: min 1 Design mandated, max 9 total, $D+A \leq 3$; see [team-formation.pdf](#)). If the visual/material polish per floor outgrows one artist, trade the 2nd designer for a 2nd artist. Note: for Immersive Experience the audio floor is lighter (ambient + UI cues) — a lecturer waiver on heavy BGM may apply for a utility wayfinding app.

Track goals — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the indoor-wayfinding concept, target users incl. wheelchair/accessibility, the destination → mobility → multi-floor route experience, feasibility against the LiDAR/engine plan)
- M1D2 Prototype — one interactive scene/moment working end to end (select destination → see the 2.5D/3D route across floors)
- M1D3 User Research, Personas & Competitive Analysis (SIT students + accessibility-needs personas; teardown of Google Indoor Maps / Mappedin / Matterport; findings folded into the FDD)
- M1D6 Experience Flow & Pacing (the scan → search → route → walk-view flow, floor-transition legibility, dwell/decision points)
- M1D7 Reactivity Model (mapping live crowd density / obstruction reports to routing responses — reroute thresholds, warning states — as tunable parameters)

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D5 Playtesting & Iteration Program (route-following + accessibility-mode tests with real students, triaged)
- M2D6 Vertical Slice — one complete floor/route polished end to end (destination → accessible route → arrival)

- M2D7 Reactivity Tuning (the crowd-density / obstruction thresholds that trigger rerouting, tuned from observation)
- **UI/UX M1** (graded by the UI/UX instructor):
 - M1U1 ★ UI Standards & Wireframe Baseline — the bottom-tab shell, destination search, mobility-preference and route-view standards; readable over a busy 3D scene.
 - M1U2 UX Flows & Wireframes — entry → search → route → walk-view → report-obstruction, wireframed end to end.
 - M1U3 Accessibility & Readability Foundations — contrast, type scale, touch targets and input alternatives (critical given the accessibility mandate).
- **UI/UX M2:**
 - M2U1 ★ Functional UI & Usability — search, route, profile, report and settings navigable unaided on the Native-Android target.
 - M2U2 ★ UX Validation & Final Polished UI — validated, polished route + accessibility UX tested with students on the Pixel 9a.
 - M2U4 Onboarding & Tutorial Design — teach destination → mobility → route (and reporting) in the first run without a wall of text.
 - M2U5 Mobile UX Adaptation & Ergonomics — thumb-zone layout, one-handed use and interruption handling for on-the-move wayfinding, device-tested on the Pixel 9a.
- **Art M1:** 1 artist (geometry is LiDAR-captured, so style + iconography + UI are the art payload)
 - M1A1 ★ Art Bible / Proof of Concept (the campus visual direction — palette, material/lighting look for the 2.5D/3D view, wayfinding iconography)
 - M1A3 Environment, Props & VFX Concepts (the campus material/lighting styling, route/marker and obstruction VFX)
 - M1A4 UI Art Concepts & Typography (bottom-tab shell, destination/mobility UI, route + accessibility iconography)
- **Art M2:**
 - M2A1 ★ Critical Asset Production & In-Engine Visual Quality (the campus look, wayfinding icons and UI implemented and consistent)

- M2A3 Environment Alive — Ambient Animation & Layering (lighting/ambient pass so the 3D campus reads clearly per floor)
- M2A4 UI Art — States & Feedback (routing / rerouting / obstruction-reported / accessibility-mode states)
- **TEAM M1:**
 - M1TA1 ★ Overall Audio Experience — initial audio from the week-5 needs list present & working (subtle ambient + UI/navigation cues: destination set, reroute, arrival), suitable and correctly triggered (a lecturer waiver on BGM may apply for this utility).
 - M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback.
- **TEAM M2:**
 - M2TA1 ★ Overall Audio Experience — the Immersive-Experience floor adapted to a wayfinding utility (ambient + navigation/arrival/reroute/report cues, ≥ the agreed set), correctly triggered.
 - M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the playback functionality the build needs (multi-channel; a distinct reroute/arrival cue).
 - M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the conversion tool + engine inspector), prototype demo, team post-mortem (wk14)

*Generated by the Project Proposal Assessor skill · grounding: live folder
(Project 2 Technical adjustments/) · references verified 2026-09-21.*